

← Go Back

Hardware

UNO R4 WiFi

Tutorials

- Arduino UNO R4 WiFi ADC Resolution
- Arduino UNO R4 WiFi CAN Bus
- Arduino UNO R4 WiFi User Manual
- Arduino UNO R4 WiFi Arduino Cloud Setup Guide
- Arduino UNO R4 WiFi Digital-to-Analog Converter (DAC)**
- Debugging the Arduino UNO R4 WiFi
- Arduino UNO R4 WiFi EEPROM
- UNO R4 WiFi Custom Firmware Upload to ESP32 (Advanced)
- Using the Arduino UNO R4 WiFi LED Matrix
- Arduino UNO R4 WiFi OPAMP
- Arduino UNO R4 WiFi Qwiic Connector
- Getting Started with UNO R4 WiFi
- Arduino UNO R4 WiFi Real-Time Clock
- UNO R4 WiFi Stack Trace Debug Runtime Errors
- UNO R4 Capacitive-Touch Tutorial

Home / Hardware / UNO R4 WiFi / **Arduino UNO R4 WiFi Digital-to-Analog Converter (DAC)**

Arduino UNO R4 WiFi Digital-to-Analog Converter (DAC)

Learn how create waveforms and output them on a piezo, using the DAC on the UNO R4 WiFi board.

Author · Jacob Hylén · Last revision · 10/07/2025

The Arduino UNO R4 WiFi has a built in **DAC** (Digital-to-analog Converter) which is used to transform a digital signal to an analog one. This feature can be used to build a plethora of fun audio projects, but also work as professional lab equipment as a cheap function generator, for example.

Goals

In this article, you will learn:

- About the DAC feature onboard the UNO R4 WiFi,
- differences between PWM and DAC techniques,
- how to generate a waveform (sine),
- how to output this waveform on a piezo speaker.

ON THIS PAGE

Goals

- Hardware & Software Needed
- Circuit
- Analog Output vs PWM
- Code
- Testing It Out
- Conclusion

Hardware & Software Needed

To follow along with this article, you will need the following hardware:

Arduino UNO R4 WiFi

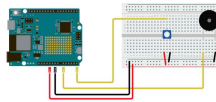
Piezo buzzer

Potentiometer

Jumper wires

Circuit

The circuit required for this tutorial can be found in the diagram below:



Piezo buzzer
connected to UNO R4

Analog Output vs PWM

For many use cases when analog output is required, using PWM (Pulse Width Modulation) instead of genuine analog output will yield essentially the same results. A digital output pin can only either be fully on (HIGH) or fully off (LOW), but by turning on and off very quickly with precise timings, the

analog output. This method is called **PWM**.

For example when dimming an LED, you can freely use a PWM enabled digital pin as an analog output pin and the LED would dim just the same as if you'd be using a DAC output.

However this will not always be the case, and for many uses you will need to use a genuine analog output to get your desired results. One such case is in audio purposes, where a PWM output simply will not give the same quality of sound as a genuine analog output, and requires some fiddling to work in the first place.

Code

The code for this tutorial is split in two parts, one main sketch and a header file containing a pre-generated sine waveform.

With this sketch, we have pre-generated a sine waveform. You could also dynamically generate it either at the beginning of your sketch or during, but doing so would be less efficient without gaining any performance. So going this route is the best practice.

The waveform is being stored as samples in an array, and with every loop of the sketch we'll update the DACs output value to the next value in the array.

Open a new sketch and paste the following code into your window.



Code Source on
[Github](#)

```
1  /*
2   SineWave
3
4   Generates a pre-g
5
6   See the full docu
7   https://docs.ardu
8  */
9
10 #include "analogWav
11
12 analogWave wave(DAC
13
14 int freq = 10; //
15
16 void setup() {
17   Serial.begin(11520
18   wave.sine(freq);
19 }
20
21 void loop() {
22   // Read an analog
23   freq = map(analogR
24
25   // Print the updat
26   Serial.println("F
27
28   wave.freq(freq);
29 }
```

Testing It Out

Once you have uploaded the code to the board, it should start generating a sine wave oscillation on the DAC, that depending on the frequency could be used to produce sound

hand, connecting its probe to the DAC output might be an interesting exercise so see what the wave looks like.

Now try twisting the potentiometer, and listen to how the sound changes.

Analog Wave Types

The sketch above generates what is known as a sine wave. It is called a sine wave because if you were to plot the voltage against time, you'd notice that the line looks just like a sine function.

There are other types of analog waves that will produce a distinctly different type of sound compared to a sine wave. The library we're using in this sketch also allows you to create sawtooth and square waves. These types of wave also gets their names from how they look when the voltages are plotted against time.

Try changing the wave type and listen how it changes the feel of the sound.

Change the wave by replacing

Line 18 in the sketch "

`wave.sine(freq);` " with either

```
1 wave.square(freq);
```

or

```
1 wave.saw(freq);
```

Now that you know your setup is working, you can experiment further with different examples and see how you can use the DAC of the UNO R4 to generate sounds and even melodies.

Note: In this setup, we're just using a piezo buzzer, you may notice that the sounds it's making are pretty faint. If you want to

 fix this you'll need a 4 or 8 Ohm speaker, and an amplifier. You can find many breakout amplifier modules that are easy to use online.

Frere Jacques

This one for example plays the melody of Frere Jacques:

 Code Source on [Github](#)

```
1  /*
2   DAC Melody player
3
4   Generates a series of notes
5   using the Uno R4 Mini
6   The melody is "F major"
7
8   circuit:
9     * audio amp (LM386)
10    * audio amp input
11    * 4-8-ohm speaker
12    * Potentiometer
13
14    created 13 Feb 2015
15    modified 3 Jul 2015
16    by Tom Igoe
17
18    See the full documentation at
19    https://docs.arduino.cc
20  */
21  #include "analogWave.h"
22  analogWave wave(DAC);
23
24  #define NOTE_A4 69
25  #define FREQ_A4 440
26
27  // the tonic, or first note of the scale
28  int tonic = 65;
```

MIDI Piano Notes

This sketch will break down the potentiometer input into steps, that are translated to the 88 MIDI notes that represent the keys on a piano.



Code Source on
[Github](#)

```
1  /*
2   Plays a tone in r
3   formula from http
4   and https://en.wi
5
6   the MIDI protocol
7   128 possible note
8   a formula for con
9   reduces that to t
10
11   frequency = 4.
12
13   You can see this :
14
15   circuit:
16     * audio amp (LI
17     * audio amp in
18     * 4-8-ohm speal
19     * Potentiomete
20
21   created 18 Dec 20
22   modified 3 Jul 20
23   by Tom Igoe
24
25   See the full docu
26   https://docs.ardu
27  */
28
```

Conclusion

By following this tutorials you've experimented with the DAC on the Arduino UNO R4 boards and used it to first generate a sine wave, and then to explore the possibilities of analog output by testing out various examples.

Suggest changes

The content on docs.arduino.cc is

Need support?
[Help Center](#)
[Ask the Arduino Forum](#)

License

The Arduino documentation is licensed under the [Creative Commons](#)

GitHub
repository.
If you see
anything
wrong, you
can edit
this page
here.

[Share Alike](#)
[4.0](#) license.

Was this article helpful?

